

P001 | 有限電池與變動服務窗口

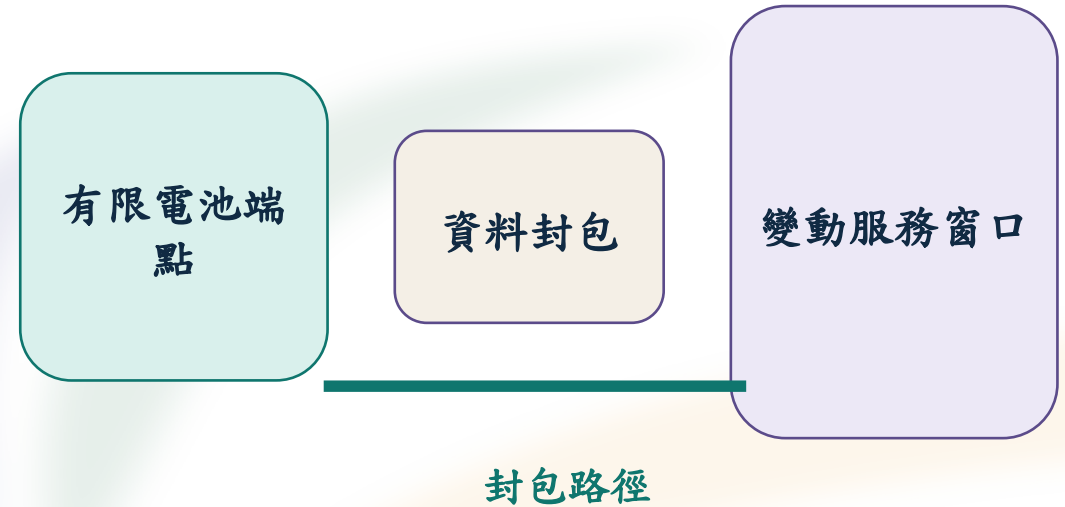
scenario（情境定義）：source = course package；
purpose = 固定端點、窗口與資料規則；unit =
identifier；interpretation = 同一組輸入供各 case 重跑。

環境感測端點在固定 scenario 內產生資料封包；端點
電池有限，服務窗口可開啟、關閉或縮短。

endpoint（端點）：source = scenario fixture；purpose
= 執行 policy；unit = classification；interpretation =
以端點 radio / processing energy model 記錄控制後果。

service window（服務窗口）：source = changing-
service-window trace；purpose = 定義可傳輸區間；
unit = trace time；interpretation = 決定 action 是否具
有合法服務機會。

觀察順序：窗口狀態 → policy action → packet /
service 結果 → endpoint energy。



P002 | Action 的三條能量路徑

action（策略動作）：source = policy API；purpose = 每次決策的唯一輸出；unit = enum；interpretation = 觸發 runner 的狀態與封包轉移。

WAIT（清醒等待）：source = policy API；purpose = 保持無線端點可反應；unit = enum；interpretation = 累積清醒閒置能量。

SLEEP（低功耗休息）：source = policy API；purpose = 降低休息功率；unit = enum；interpretation = 產生喚醒延遲與喚醒能量。

SEND_ONE（單筆傳輸）、*SEND_URGENT*（緊急傳輸）、*FLUSH_BATCH*（批次送出）：source = policy API；purpose = 送出封包；unit = enum；interpretation = 影響處理、發射、接收、重試與服務。

同一個窗口內，action 的選擇同時改變服務機會與 endpoint energy。

清醒等待

低功耗休息

送出封包

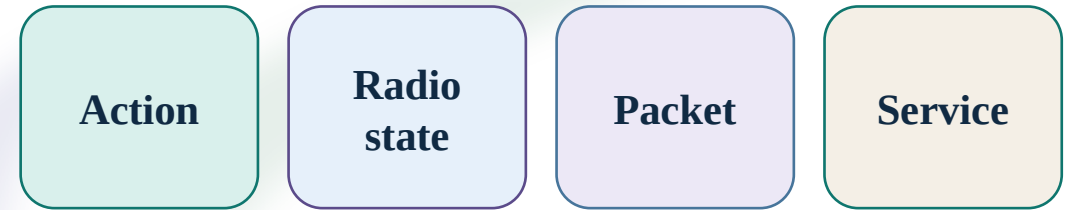
P003 | Action 連接狀態、封包與服務

radio state（無線狀態）：source = runner event ledger；purpose = 表示休眠、清醒閒置、處理、發射與接收；unit = state enum + duration；interpretation = 提供端點能量的時間分段。

packet progress（封包進度）：source = result / replay artifact；purpose = 記錄產生、佇列、嘗試、重試、送達與逾期；unit = event count / time；interpretation = 連接 action 與服務結果。

service result（服務結果）：source = scenario deadline / freshness rule；purpose = 判定資料是否在規則內完成；unit = pass / fail + reason；interpretation = energy comparison 的邊界條件。

窗口關閉時，傳輸類 action 不進入 runner；安全分支為 *SLEEP*。



P004 | 固定情境中的改、跑、讀、解釋

policy（策略）：source = marked block in *student_policy.py*；purpose = 依觀測輸入回傳合法 action；unit = code contract；interpretation = 唯一受控決策面。

JSON（結構化資料格式）：source = package runner；purpose = 保存 machine-readable result；unit = data format；interpretation = 讓 result、replay 與 workbook 可核對。

result.json（結果資料檔）：source = package runner；purpose = 保存 run identity、state、packet、service、energy；unit = JSON artifact；interpretation = 後續 replay / import 的輸入。

操作閉環：改一個 bounded constant → 執行固定 case → 讀事件與結果 → 以 policy → action → event → service / energy 說明差異。

prediction 與 result 同時保存；反例保留為可檢驗的 evidence。

改一個值

執行 case

讀取事件

解釋差異

P005 | 因果鏈的讀法：中間事件決定歸因

observation（觀測輸入）：source = runner step；purpose = 提供目前窗口、品質、佇列與期限；unit = schema fields；interpretation = policy 可使用的當下資訊。

policy（策略）與 *action*（動作）：source = student_policy.py + policy API；purpose = 由觀測選出合法控制；unit = code / enum；interpretation = 變更來源。

state（狀態）與 *packet*（封包）：source = endpoint replay；purpose = 呈現狀態區間與傳輸事件；unit = duration / count / time；interpretation = 中間機制。

service（服務判定）與 *endpoint energy*（端點能量）：source = scenario rule + endpoint model；purpose = 判讀傳輸完成與累積焦耳；unit = pass / fail 與 J；interpretation = 結果層。

完整順序：observation → policy → action → state / packet → service → endpoint energy。

觀測

策略

動作

中間事件

服務／能量

P006 | 可重跑 Runner 的證據範圍

runner（執行器）：source = course package；purpose = 以固定 scenario、policy 與 seed 產生可重跑 artifact；unit = executable package；interpretation = controlled teaching evidence。

seed（固定隨機種子）：source = scenario contract；purpose = 固定隨機序列；unit = integer identifier；interpretation = 相同輸入可重現事件順序。

endpoint_energy_j（端點能量）：source = endpoint radio / processing model；purpose = 累積狀態能量；unit = J；interpretation = endpoint evidence layer。

coherent simulated result（連貫模擬結果）：source = course runner；purpose = 連結 policy 與事件；unit = evidence classification；interpretation = endpoint / service 教學資料。

Endpoint energy、LEO / system energy 與 canonical/system efficiency 保持不同欄位與不同 claim scope。

固定 scenario

固定 seed

事件 ledger

endpoint J

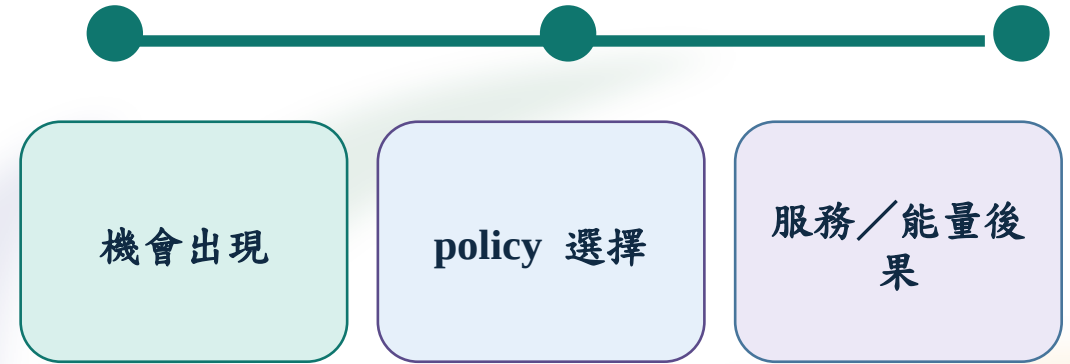
P007 | LEO 作為變動服務窗口範例

LEO（低地球軌道）：source = fixed course scenario trace；purpose = 提供可開、可關、可變品質的 service-window input；unit = scenario example；interpretation = changing opportunity example。

LEO 的作用是改變服務機會；學習焦點是端點 policy 如何在窗口變化下選擇傳輸、等待或休息。

HVAC（暖通空調）：source = transfer example；purpose = 代表低負載或可連線時段；unit = application context；interpretation = 可轉移的 changing-window 機制。

edge gateway（邊緣閘道）與物流追蹤可沿用同一機制；各情境重新定義窗口、資料期限與 energy boundary。



P008 | 預測到結果：保留可反駁路徑

prediction（預測）：source = workbook record；purpose = 指定 state、packet、service 或 endpoint energy 的可反駁方向；unit = statement；interpretation = run 前的比較基準。

workbook（決策工作簿）：source = course route；purpose = 保存 prediction、run、replay 與 interpretation；unit = structured record；interpretation = 可重開的證據脈絡。

withheld case（保留條件案例）：source = frozen scenario package；purpose = 以未調整的 policy 檢驗轉移性；unit = case label；interpretation = counterexample evidence。

result_path（結果檔路徑）：source = run command output；purpose = 指向 result.json；unit = path string；interpretation = import / replay 配對入口。

流程：記錄 prediction → bounded edit → exact run → 比較 state / packet / service / energy → 保存反例。

prediction

bounded edit

exact run

withheld

P009 | Provenance 與 Claim Ceiling

provenance（來源鏈）：source = package receipt、scenario、policy identity、run artifact；purpose = 連結輸入與輸出；unit = structured record；interpretation = 追溯 evidence scope。

source_mode（來源模式）：source = result metadata；purpose = 區分 *student-run* 與 *same-scenario-fallback*；unit = enum；interpretation = 來源分類保持原值。

claim_boundary（主張邊界）：source = course contract；purpose = 標示 evidence class；unit = fixed string；interpretation = *SIMULATED TEACHING DATA / NOT LIVE / NOT MEASURED / NOT CANONICAL-PARITY-VERIFIED*。

package identity（套件識別）：source = release manifest；purpose = 綁定 package 與 artifact；unit = identifier；interpretation = identity mismatch stops import。

結果解讀依 *source_mode*、scenario identity、policy identity 與 *claim_boundary* 一起進行。

來源鏈

主張邊界

可比較欄位

P010 | Release asset 與 package root

release asset（發布封裝）：source = course package distribution；purpose = 提供同一組 runner files；unit = archive file；interpretation = package identity boundary。

lora-energy-lab-v1.zip（課程封裝檔）：source = release page；purpose = transport package root；unit = file；interpretation = 解壓後形成單一 *lora-energy-lab/* 根目錄。

README.zh-TW.md、*setup.sh*、*setup.cmd*、*course.sh*、*course.cmd*、*student_policy.py*：source = package root；purpose = 說明、設定、驗證、執行與 policy；unit = files；interpretation = required surface。

schemas/（結構契約目錄）與 *fallback_artifacts/*（同情境回復資料目錄）：source = package root；purpose = validate and recover；unit = directories；interpretation = preserve scenario identity。

lora-energy-lab/

setup / course

student_policy.py

schemas / fallback

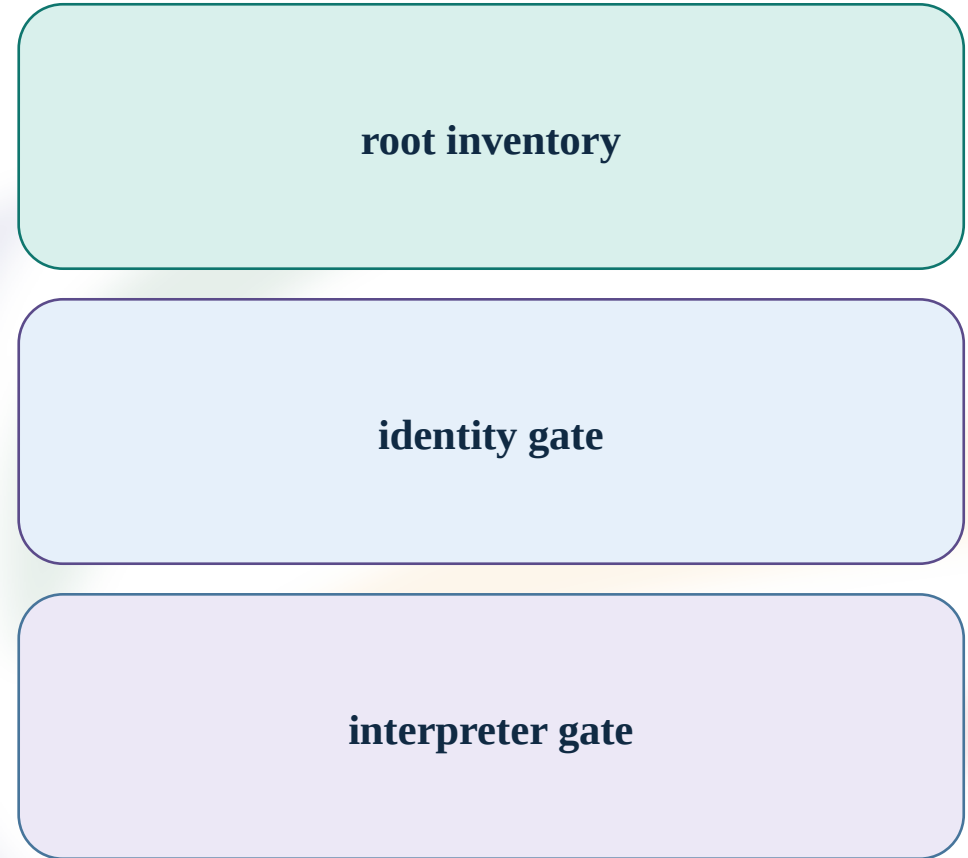
P011 | Package identity gate 與 root inventory

package identity（套件識別）：source = release manifest + root inventory；purpose = verify required files are one coherent package；unit = identifier/list；interpretation = identity mismatch stops setup。

package root（套件根目錄）：source = extracted release asset；purpose = anchor setup.sh / setup.cmd and course.sh / course.cmd；unit = directory；interpretation = .venv and artifacts stay local。

Required inventory：README、兩套 setup / launcher、*student_policy.py*、schemas、fallback _artifacts。

Gate result：root complete → interpreter gate；root incomplete → 保留錯誤並重新取得同一 release asset。



P012 | Windows shell identity 與 Python 3.11

CMD (Windows 命令提示字元) : source = Windows native shell ; purpose = execute .cmd launcher ; unit = shell type ; interpretation = path uses \ 。

PowerShell (Windows 管理命令殼) : source = Windows native shell ; purpose = execute .cmd and inspect receipt ; unit = shell type ; interpretation = command syntax differs from CMD 。

Python 3.11.x (固定直譯器版本) : source = runner frozen contract ; purpose = create package-local .venv ; unit = version ; interpretation = version gate before setup 。

Commands : *py --list* ; *py -3.11 --version* → expected *Python 3.11.x* 。



P013 | Windows interpreter recovery

uv (Python 執行環境管理工具) : source = official runtime manager ; purpose = obtain and locate exact Python 3.11 ; unit = tool / version / path ; interpretation = interpreter recovery only 。

uv python install 3.11 : source = uv tool ; purpose = install exact minor ; unit = command ; interpretation = creates available interpreter 。

uv python find 3.11 : source = uv tool ; purpose = return interpreter path ; unit = path string ; interpretation = path passes to setup 。

Recovery result is an interpreter path ; package setup and *READY* receipt remain later gates 。

取得 uv

安裝 3.11

定位 path

進入 setup

P014 | POSIX / WSL shell separation

POSIX (Unix-like shell 環境) : source = macOS / Linux / WSL ; purpose = run *.sh* ; unit = environment class ; interpretation = path uses / 。

WSL (Windows Subsystem for Linux ; Windows 的 Linux 執行層) : source = host platform ; purpose = run Ubuntu / Linux tools ; unit = environment class ; interpretation = owns a separate *.venv* 。

Ubuntu (Linux 發行版) : source = WSL distribution ; purpose = provide verified Linux shell ; unit = distribution name ; interpretation = *setup.sh* and *course.sh* run inside this boundary 。

Identity check : *pwd* shows Linux path ; *uname -a* begins with Linux 。



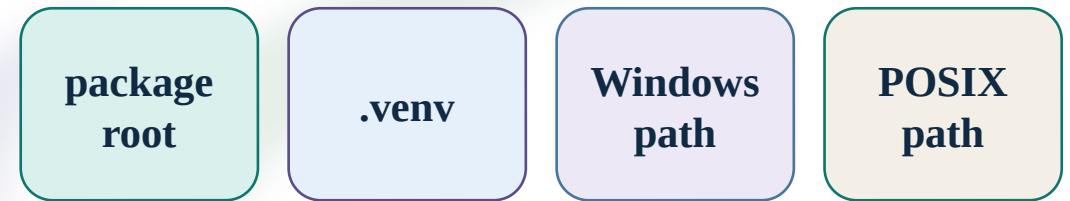
P015 | package-local .venv 隔離

.venv（套件本地虛擬環境）：source = Python *venv* module；purpose = isolate locked dependencies and policy；unit = directory；interpretation = launcher uses its own interpreter。

venv（虛擬環境標準模組）：source = Python standard library；purpose = create *.venv*；unit = module；interpretation = environment creation mechanism, not an interpreter version。

Windows path：.venv\Scripts\python.exe；POSIX path：.venv/bin/python。

Package-local Python should report *Python 3.11.x*；啟用只作診斷，正常 launcher 直接呼叫 package-local interpreter。



同一 platform boundary

P016 | Windows setup 與 READY receipt

setup.cmd (Windows 設定啟動器) : source = package root ; purpose = create .venv, install locked dependencies, run verification ; unit = executable file ; interpretation = prepares package boundary 。

course.cmd verify (Windows 驗證命令) : source = package launcher ; purpose = re-check interpreter / lock / scenario / policy contract ; unit = command ; interpretation = emits verification receipt 。

READY (環境就緒狀態) : source = *artifacts/verify-receipt.json* ; purpose = indicate environment / contract gate passed ; unit = enum ; interpretation = runner may enter case execution 。

Expected fields : status READY 、 Python 3.11.x 、 scenario identity 、 engine mode 、 claim boundary 。

setup.cmd

course.cmd verify

READY

verify receipt

P017 | WSL identity gate

`wsl --install -d Ubuntu` (WSL 安裝命令) :
source = Windows PowerShell ; purpose = create
Ubuntu distribution when absent ; unit =
command ; interpretation = host setup only 。

`wsl -l -v` (WSL 分發清單) : source = Windows
command ; purpose = show distribution / version
/ status ; unit = list ; interpretation = selects
correct Ubuntu boundary 。

`wsl -d Ubuntu` (進入命令) : source = Windows
command ; purpose = open Ubuntu shell ; unit =
command ; interpretation = subsequent `.sh`
commands belong to Linux 。

Ubuntu shell evidence : `pwd` Linux path and `uname -a` Linux kernel label 。

Windows shell

wsl -l -v

Ubuntu

Linux shell

P018 | WSL toolchain ladder

apt (Linux 套件管理器) : source = Ubuntu ; purpose = obtain *curl* and *unzip* ; unit = command / tool ; interpretation = provides release extraction prerequisites 。

curl (命令列傳輸工具) : source = Ubuntu package ; purpose = retrieve uv installer ; unit = tool ; interpretation = environment setup input 。

unzip (封裝解壓工具) : source = Ubuntu package ; purpose = expand release asset ; unit = tool ; interpretation = creates package root 。

Order : apt update / install → uv version / install / find → Python 3.11 path 。

apt

curl / unzip

uv

Python 3.11

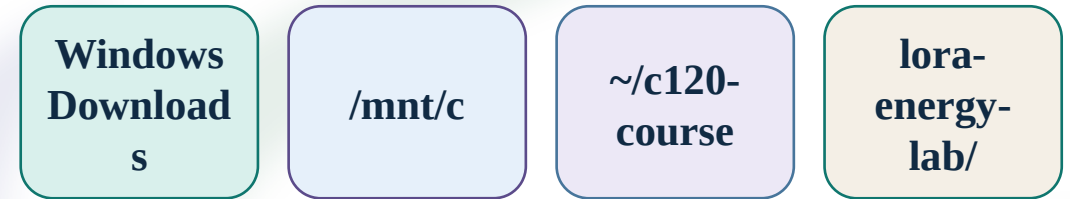
P019 | WSL package path

`/mnt/c/.../Downloads` (Windows mount path) :
source = WSL mount ; purpose = read release asset
from Windows Downloads ; unit = filesystem
path ; interpretation = transfer source only 。

`~/c120-course` (Linux course staging path) :
source = Ubuntu home ; purpose = hold release files
and extracted root ; unit = directory ; interpretation
= Linux-owned `.venv` boundary 。

`cp` (複製命令) : source = POSIX shell ; purpose
= copy release files into Linux home ; unit =
command ; interpretation = prevents cross-platform
environment mixing 。

`pwd` (目前目錄命令) : source = POSIX shell ;
purpose = confirm package root ; unit = path
string ; interpretation = setup runs from `lora-energy-
lab/` 。



同一 platform boundary

P020 | POSIX / WSL setup 與 verify

setup.sh (POSIX 設定啟動器) : source = package root ; purpose = create *.venv*, install lock dependencies, call verification ; unit = executable file ; interpretation = prepares Linux / macOS / WSL runner 。

course.sh verify (POSIX 驗證命令) : source = package launcher ; purpose = check exact interpreter and course contract ; unit = command ; interpretation = writes *artifacts/verify-receipt.json* 。

PYTHON_BIN (Python 執行器路徑變數) : source = shell environment ; purpose = select *python3.11* or *uv python find 3.11* output ; unit = path string ; interpretation = setup input 。

Expected status : READY ; Python 3.11.x ; package-local *.venv/bin/python* 。

PYTHON_BIN

setup.sh

course.sh verify

READY

P021a | 查看 READY receipt

verify-receipt.json (驗證收據檔) : source = setup / verify command ; purpose = record environment and contract gate ; unit = JSON artifact ; interpretation = setup evidence 。

POSIX : *sed -n '1,80p' artifacts/verify-receipt.json*

Windows PowerShell : *Get-Content .\artifacts\verify-receipt.json*

Windows CMD : *type artifacts/verify-receipt.json*

Read fields as structured data ; preserve file provenance 。

POSIX

PowerShell

CMD

receipt

P021b | READY receipt : 狀態與識別欄位

status (狀態) : source = receipt ; purpose = gate result ; unit = enum ; interpretation = READY permits case entry 。

python_version (Python 版本) : source = receipt ; purpose = record interpreter ; unit = version ; interpretation = must be 3.11.x 。

scenario_id (情境識別碼) : source = scenario contract / receipt ; purpose = bind fixed case ; unit = string identifier ; interpretation = identifies the scenario, not an energy result 。

policy_api_version (策略介面版本) : source = package contract ; purpose = bind allowed observations / actions ; unit = string identifier ; interpretation = prevents API drift 。

P021c | READY receipt : 引擎與主張欄位

engine_mode (執行引擎模式) : source = runner metadata ; purpose = classify producer ; unit = enum ; interpretation = *coherent-course-simulated-adapter* 。

upstream_execution (上游程式執行狀態) : source = runner metadata ; purpose = state whether upstream repo ran ; unit = Boolean ; interpretation = false for course adapter 。

claim_boundary (主張邊界) : source = course contract ; purpose = label evidence ; unit = fixed string ; interpretation = *SIMULATED TEACHING DATA / NOT LIVE / NOT MEASURED / NOT CANONICAL-PARITY-VERIFIED* 。

Receipt 通過表示環境與契約一致； case result 仍需另行產生並以自己的 run identity 保存。

P022 | Setup gate recovery 與 same-scenario fallback

artifact_source（資料來源分類）：source = result metadata；purpose = label local policy run or fallback；unit = enum；interpretation = source mode remains visible。

same-scenario-fallback（同情境回復資料）：source = package *fallback_artifacts/*；purpose = continue a case when local setup is blocked；unit = artifact pair；interpretation = same scenario identity, no local policy execution claim。

fallback_artifacts/<case>/result.json（回復結果檔）與 *endpoint-replay.json*（端點重播檔）：source = package artifact；purpose = preserve result / replay pairing；unit = JSON files；interpretation = imported together。

Recovery decision：修復指示的 boundary 或使用 exact case pair；保留錯誤與來源分類。

環境 gate

修復 boundary

same-scenario-
fallback

P023 | Bounded edit surface : student_policy.py

student_policy.py (策略檔名) : source = package root ; purpose = expose three marked blocks ; unit = Python file ; interpretation = only editable course surface 。

marked block (標記區段) : source = file markers ; purpose = bound one lab edit ; unit = source range ; interpretation = keeps policy identity and result lineage interpretable 。

lab-a-pace-rest 、 *lab-b-enter-exit-hold* 、 *lab-c-batch-urgent* : source = marked block labels ; purpose = select current lab control ; unit = identifier ; interpretation = one active region per edit 。

Scenario 、 schemas 、 runner 、 網站程式與 generated artifact remain read-only package inputs / outputs 。

student_policy.py

A marked block

B marked block

C marked block

P024a | Observation 是 decision-time input

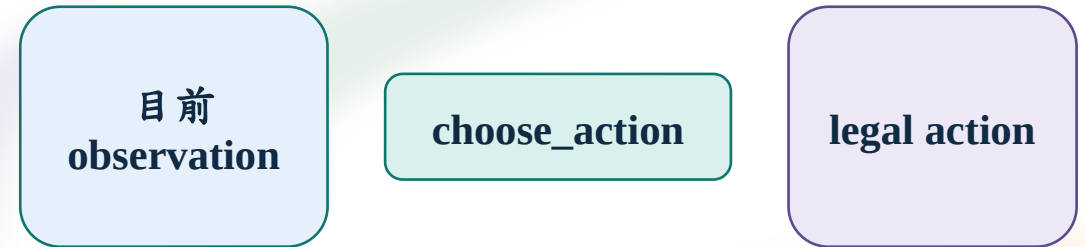
choose_action(observation) (策略函式) : source = policy API ; purpose = read current observation and return one legal action ; unit = function contract ; interpretation = no future / result access 。

contact_open (窗口開啟狀態) : source = current scenario observation ; purpose = guard send legality ; unit = Boolean ; interpretation = false routes to SLEEP 。

quality_band (品質分帶) : source = current trace observation ; purpose = expose present link quality category ; unit = ordinal band ; interpretation = threshold input 。

stable_steps (穩定步數) : source = trace accumulator ; purpose = count consecutive same-band observations ; unit = steps ; interpretation = hold gate input 。

These are decision-time inputs ; future quality 、 future energy 與 result summary outside policy 。



P024b | Observation 欄位與合法 action

steps_since_send（距上次送出步數）： source = runner state ； purpose = pacing input ； unit = steps ； interpretation = guards rest gap 。

queue_size（佇列數量）： source = endpoint queue ； purpose = batching / urgency input ； unit = packet count ； interpretation = determines send pressure 。

urgent_due_in_s（緊急期限剩餘秒數）： source = scenario deadline ； purpose = urgency input ； unit = 秒（ s ） ； interpretation = compares with urgency margin 。

previous_action（前一動作）： source = runner state ； purpose = detect transitions ； unit = action enum ； interpretation = continuity input 。

contact_remaining_s（窗口剩餘秒數）： source = contact trace ； purpose = opportunity input ； unit = 秒（ s ） ； interpretation = informs send timing 。

合法輸出： *WAIT* / *SLEEP* / *SEND_ONE* / *SEND_URGENT* / *FLUSH_BATCH* 。

queue /
deadline

contact / pace

合法輸出

P025a | Lab A / B 的受控常數

PACE_GAP_STEPS（節奏間隔步數）：source = Lab A marked block；purpose = minimum steps between sends；unit = steps；interpretation = pacing boundary。

REST_DURING_GAP（空檔休息動作）：source = Lab A marked block；purpose = choose WAIT or SLEEP during gap；unit = action enum；interpretation = changes awake-idle versus wake cost。

ENTER_QUALITY（進入品質門檻）：source = Lab B marked block；purpose = activate send mode；unit = quality band；interpretation = enter threshold。

EXIT_QUALITY（退出品質門檻）：source = Lab B marked block；purpose = deactivate send mode；unit = quality band；interpretation = exit threshold。

STABLE_STEPS（穩定步數門檻）：source = Lab B marked block；purpose = require consecutive stable observations；unit = steps；interpretation = hold control。

hysteresis（遲滯控制）：source = enter / exit / hold constants；purpose = separate thresholds and hold duration；unit = rule；interpretation = reduces ping-pong transitions。

PACE_GAP_STEPS

REST_DURING_GAP

ENTER /
EXIT_QUALITY

STABLE_STEPS

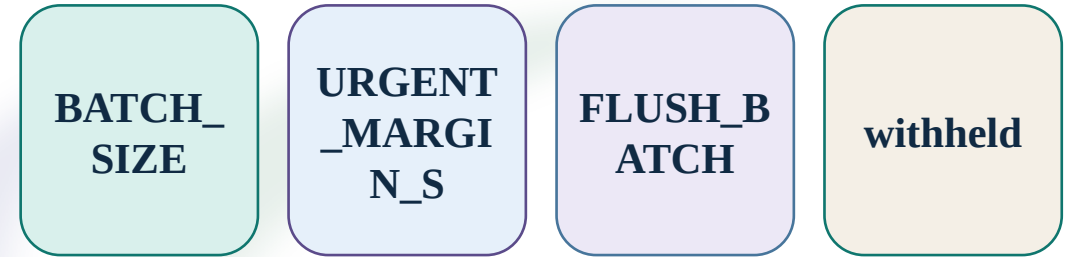
P025b | Lab C 的佇列與緊急常數

BATCH_SIZE（批次大小）：source = Lab C marked block；purpose = set packets grouped for flush；unit = packet count；interpretation = controls batching and wait energy。

URGENT_MARGIN_S（緊急餘裕秒數）：source = Lab C marked block；purpose = trigger urgent action before deadline；unit = 秒（s）；interpretation = changes deadline protection。

FLUSH_BATCH（批次送出）：source = policy API；purpose = transmit queued packets as a group；unit = action enum；interpretation = links queue to process / TX / RX and delivery。

Candidate、revision 各改一個 marked constant；withheld surprise 維持 frozen policy。



一次一個受控常數

P026a | 讀取條件與 action 分支

Policy branch order is evaluated from current observation 。

contact_open = false → *SLEEP* : source = policy API ;
purpose = keep send illegal outside window ; unit =
Boolean → enum ; interpretation = protects service
legality 。

urgent_pending (緊急封包待處理) : source = queue /
deadline observation ; purpose = reveal urgent packet ;
unit = Boolean ; interpretation = activates deadline
branch 。

urgent_due_in_s ≤ *URGENT_MARGIN_S* →
SEND_URGENT : source = deadline + policy constant ;
purpose = protect urgent deadline ; unit = 秒比較 ;
interpretation = earlier trigger as margin grows 。

steps_since_send < *PACE_GAP_STEPS* →
REST_DURING_GAP : source = pacing observation +
constant ; purpose = hold rest behavior ; unit = steps
comparison ; interpretation = output can be WAIT or
SLEEP 。

窗口

緊急期限

節奏間隔

action

P026b | Action 後果的事件讀法

WAIT → 清醒閒置 interval ; source = runner ledger ; purpose = retain response ; unit = duration / J ; interpretation = time-integrated energy cost 。

SLEEP → 休眠 interval + wake transition ; source = runner ledger ; purpose = reduce idle power ; unit = duration / J ; interpretation = wake latency / energy appear in event chain 。

*SEND_** → process / TX / RX / attempt / retry / delivery ; source = endpoint replay ; purpose = show packet path ; unit = duration / count ; interpretation = connects action to service 。

endpoint_energy_j 與 service result 一起判讀；較低 J 且服務失敗時，分類為 energy / service trade-off 。

**WAIT /
SLEEP**

**state
interval**

SEND_*

**service /
endpoint
J**

P027a | Python indentation 與 return

indentation (縮排) : source = Python syntax ; purpose = assign statements to branch ; unit = code structure ; interpretation = determines branch membership 。

return (回傳) : source = policy function ; purpose = send one legal action to runner ; unit = action enum ; interpretation = closes decision step 。

Minimal skeleton : *def choose_action(observation):* → *if not observation.contact_open:* → *return SLEEP* 。

Syntax gate failure is repaired inside the active marked block ; runner 、 scenario 、 schema 維持原狀。

observation
↓ branch

return action

P027b | Lab A baseline run 與結果入口

baseline（基線執行）：source = packaged policy + fixed scenario / case；purpose = provide control artifact；unit = case label；interpretation = comparison anchor。

bash course.sh run --lab A --case baseline / course.cmd run --lab A --case baseline：source = package launcher；purpose = execute exact case；unit = command；interpretation = emits result path and run identity。

result_path（結果檔路徑）：source = command JSON output；purpose = locate *result.json*；unit = path string；interpretation = import and replay pair entry。

endpoint-replay.json（端點重播檔）：source = same run directory；purpose = replay action / state / queue / packet timing；unit = JSON artifact；interpretation = event interpretation source。

Expected status *OK* is an artifact status；claim boundary remains course-simulated。

baseline

result.json

endpoint-replay.json

run identity